VR Application

Full-Stack 3D Web Application - Technical Report

Candidate Project	Techdojo VR Application technical round implementation
Repository	https://github.com/SamirMahmud28/vr-application
Live Demo	https://vr-application-sigma.vercel.app
Backend Health Check	https://vr-application-backend.onrender.com/api/test
Estimated Completion	95% - all core requirements completed; remaining work is optional UI polish and production hardening.

1. Project Overview

This project is a full-stack 3D web application developed for the Techdojo technical round. The application allows users to sign up, log in using session-based authentication, enter a 3D scene, add multiple objects, drag objects in 3D space, save the scene state to MongoDB, and load the saved scene again after login.

The solution was implemented incrementally with regular Git commits to keep the development history clear for evaluation. The final deployment uses Vercel for the React frontend, Render for the Express backend, and MongoDB Atlas for database persistence.

Requirement Alignment Summary

- Authentication: signup, login, logout, session tracking, and MongoDB-backed session storage.
- 3D scene: room environment with camera, lighting, floor/grid, and visible UI controls.
- Object system: cube, sphere, and custom GLB models can be added multiple times.
- Interaction: objects can be dragged and moved on the scene floor with mouse pointer events.
- Persistence: scene objects and positions are saved to MongoDB and loaded after login.

2. Architecture

The system is structured as a separated full-stack application with a React/Vite frontend and a Node.js/Express backend. The frontend communicates with the backend API through a Vercel rewrite proxy so session cookies work reliably in the deployed environment.

Layer	Technology	Responsibility
Frontend	React, Vite, React Router, Axios	Login/signup pages, scene page, UI controls, API calls, deployed on Vercel.
3D Engine	Three.js, React Three Fiber, Drei	Canvas, camera, lighting, room setup, cube/sphere rendering, GLB model loading, object dragging.
Backend	Node.js, Express.js, Mongoose	Authentication APIs, scene save/load APIs, session middleware, deployed on Render.
Database	MongoDB Atlas	Stores users, sessions, and user-specific saved scene objects.
Deployment	Vercel + Render + MongoDB Atlas	Free-tier friendly deployment setup for interview/demo usage.

High-Level Request Flow

User Browser	Vercel Frontend	Vercel Rewrite Proxy	Render Express API	MongoDB Atlas
Signup / Login / Scene actions	React UI + Axios calls to /api	Forwards /api requests to backend	Session, auth, scene controllers	Users, sessions, scenes

Scene Data Model

Each user has one saved scene document. The scene stores an array of objects with objectId, type, position, rotation, scale, and modelUrl. This supports primitive objects now and can be extended later for additional GLB assets or transformation tools.

```
{
userId: ObjectId,
objects: [
{ objectId, type, position: [x, y, z], rotation, scale, modelUrl }
]
}
```

3. Completed Features

Requirement Area	Implementation Status	Notes
Signup / Login	Completed	User account creation, login validation, bcrypt password hashing, and session-based access.
Session Tracking	Completed	express-session and connect-mongo are used to persist sessions in MongoDB.
3D Scene	Completed	React Three Fiber scene with camera, lights, walls, floor, and grid-like room environment.
Add Objects Dialog	Completed	Radio-button dialog for cube, sphere, duck GLB, and avocado GLB models.
Random Placement	Completed	Objects are placed at random X/Z coordinates inside the scene/room boundary.
Drag and Move Objects	Completed	Mouse pointer events, raycasting, and a floor drag plane update object positions.
Save Scene	Completed	Save button sends the current object array to the backend and stores it in MongoDB.
Load Scene	Completed	Saved scene reloads after login or browser refresh.
Logout	Completed	Session is destroyed and user is redirected to login.

Key UX Details

- Save, Add Objects, object count, and Logout controls are visible inside the scene.
- Objects provide visual feedback while hovering and dragging.
- Scene messages communicate loading, saving, and successful actions.
- Deployment uses a Vercel API proxy to keep the live session flow stable.

4. Challenges, Learnings, and AI Usage

Challenges Encountered

- Session-based authentication across deployed frontend/backend domains: solved with secure cookie settings, CORS configuration, and a Vercel rewrite proxy.
- Object dragging in a 3D scene: solved using pointer events, raycasting, and a horizontal drag plane to calculate updated X/Z positions.
- Saving user-specific scenes: solved with a Scene model linked to the logged-in session user.
- GLB model scale differences: fixed by applying custom scale and height values for each GLB model type.
- Keeping the implementation interview-friendly: code was kept modular with separate services, controllers, routes, and React scene components.

New Knowledge and Skills Learned

- React Three Fiber scene setup, camera controls, lighting, and mesh rendering.
- Loading and scaling GLB/GLTF models inside a React application.
- 3D drag logic using Three.js raycasting concepts.
- MongoDB-backed Express sessions and user-specific persistence.
- Full-stack deployment using Vercel, Render, and MongoDB Atlas.

AI Tool Usage

AI assistance was used as a guided development assistant for planning sprints, interpreting the task requirements, structuring the implementation, debugging 3D interaction issues, preparing deployment changes, and drafting professional submission documentation. All generated code was tested locally and committed through Git before deployment.

Final Submission Links

GitHub Repository	https://github.com/SamirMahmud28/vr-application
Live Demo	https://vr-application-sigma.vercel.app
Backend Health Check	https://vr-application-backend.onrender.com/api/test
Completion Estimate	95% completed. Core requirements are implemented and deployed; only optional polish remains.

Conclusion

The project meets the core technical requirements for the VR Application task: authentication, 3D scene interaction, object creation, drag movement, MongoDB persistence, loading saved scenes, GitHub commit history, and live deployment.